

API Sentinel

API Cost, Abuse & Reliability Control Platform

Technical Specification & Prototype Build Document

Version 1.0 | Prototype Scope | Java 21 + Spring Boot

Purpose

API Sentinel is a real-time API governance and protection platform that sits between client applications and external APIs. It controls traffic, quotas and spending while providing caching, resilience, usage analytics, anomaly detection and operational visibility.

Core flow

Client Application → Java Gateway → Authentication → Rate Limit → Quota → Budget → Cache → External API
→ Resilience → Usage Tracking → Cost Calculation → Observability

1. Executive Summary

Problem

Applications increasingly depend on AI, payment, maps, SMS, email and internal APIs. A software bug, compromised key, malicious client or traffic spike can create excessive requests, latency, service degradation and unexpected financial cost.

Solution

API Sentinel acts as a controlled gateway. Requests are authenticated and evaluated against rate limits, quotas and budgets before being forwarded. The system records usage, calculates estimated cost and protects against failures with timeouts, retries, circuit breakers and caching.

2. Objectives

Security

API key management, authentication, authorization, secret protection and request validation.

Traffic Control

Rate limiting, request quotas, client blocking and API enable/disable controls.

Cost Control

Request-level cost tracking, daily/monthly budgets and configurable spending thresholds.

Reliability

Timeout handling, bounded retries, circuit breaking and graceful failure.

Performance

Redis caching, indexed database queries and pagination.

Observability

Request logs, error logs, usage analytics, anomaly detection and health monitoring.

3. Prototype Scope

In scope

Authentication, applications, API registration, API keys, gateway proxy, rate limiting, quotas, budgets, usage tracking, cost calculation, basic anomaly rules, caching, timeout handling, circuit breaker, dashboard, logs and health checks.

Out of scope initially

Kubernetes, complex microservices, real payment processing, ML-based anomaly detection, enterprise IAM and large-scale multi-region infrastructure.

4. Target Users

Developer

Registers applications and APIs and consumes protected APIs.

Team Admin

Controls keys, limits, budgets, users and applications.

System Administrator

Monitors traffic, failures, anomalies and platform health.

5. Core User Flow

End-to-end

Register → Login → Create Application → Register External API → Configure Limits → Generate API Key → Send Request Through Sentinel → Validate Key → Rate Limit → Quota → Budget → Cache → External API → Record Usage → Calculate Cost → Return Response.

6. Authentication & Authorization

Technology

Spring Security with JWT.

Endpoints

POST /api/v1/auth/register; POST /api/v1/auth/login; GET /api/v1/auth/me.

Security

Passwords must never be stored in plaintext. Use a strong password hashing algorithm such as BCrypt or Argon2. Keep JWT and provider secrets outside source control.

7. Applications

Data

An application belongs to a user and represents a consuming system such as a production SaaS application.

Endpoints

POST/GET/PUT/DELETE /api/v1/applications and GET /api/v1/applications/{id}.

Important fields

id, owner_id, name, environment, status, created_at, updated_at.

8. External API Registration

Configuration

Name, base URL, provider, status, rate limit, daily quota, monthly budget, timeout and cache settings.

Entity

ApiConfiguration with id, application_id, name, base_url, provider, status, rate_limit, daily_quota, monthly_budget, timeout_ms, cache_enabled, created_at and updated_at.

9. API Keys

Flow

Client sends an API key; Sentinel hashes the presented key, looks it up, validates status and expiry, and identifies the owning application.

Security

Do not store retrievable secrets in plaintext. Provider credentials that must be forwarded should be encrypted at rest and protected by an external encryption key.

10. Gateway

Role

The gateway is the central control point for protected API requests.

Flow

Request → API key validation → API configuration → rate limit → quota → budget → cache → external API → response.

Architecture

Keep the gateway controller thin. Use GatewayService plus dedicated policy services rather than placing all logic in a controller.

11. Rate Limiting

Purpose

Controls how quickly requests may arrive.

Algorithm

Token Bucket is recommended. Example: 100 tokens capacity with tokens replenished over time.

Response

Excess requests return HTTP 429 Too Many Requests.

Scaling

A prototype can use in-memory state. Multi-instance deployments should use Redis for shared state.

12. Quotas

Definition

A quota controls total consumption over a period, unlike rate limiting which controls request velocity.

Examples

100 requests/minute; 50,000 requests/day; 1,000,000 requests/month.

13. Spending Caps

Budget

Each API can have daily and monthly spending limits.

Thresholds

80% warning; 90% critical warning; 100% block.

Purpose

Prevent a runaway workload from continuing to generate cost after the configured budget is exhausted.

14. Cost Calculation

Model

Pricing must be configurable rather than hardcoded.

Formula

For token-based APIs: $\text{cost} = \text{input_tokens} \times \text{input_price} + \text{output_tokens} \times \text{output_price}$. For request-priced APIs: $\text{request_count} \times \text{request_price}$.

15. Usage Tracking

UsageRecord

id, api_id, application_id, api_key_id, request_id, status_code, latency_ms, input_units, output_units, estimated_cost, cache_hit, created_at.

Purpose

Usage records power dashboards, budgets, cost analytics, anomaly detection and auditing.

16. Request Logging

Log fields

Timestamp, API, status, latency, request ID, cost and cache outcome.

Never log

API secrets, passwords, JWTs, payment credentials or sensitive request bodies.

17. Caching

Technology

Redis.

Flow

Request → generate cache key → Redis GET → HIT returns cached response; MISS calls external API and stores a response with TTL.

Example key

api:{apiId}:request:{hash}.

Safety

Only cache responses that are safe to reuse. Make caching configurable per API.

18. Timeout Handling

Configuration

Use explicit connection and read timeouts, for example 2 seconds connection and 10 seconds read.

Failure flow

External timeout → failure handler → record failure → controlled response.

19. Retry Policy

Retry candidates

Transient timeouts and selected 5xx responses may be retried.

Do not blindly retry

400, 401 and 403 should generally not be retried. Respect Retry-After for 429 responses.

Backoff

Use bounded exponential backoff and a small maximum retry count.

20. Circuit Breaker

States

CLOSED → repeated failures → OPEN → cooldown → HALF-OPEN → test request → CLOSED on recovery or OPEN on failure.

Technology

Resilience4j is recommended for Spring Boot.

21. Anomaly Detection

Prototype approach

Use deterministic rules rather than ML.

Examples

Current hourly traffic > 5 × historical average; 5xx rate > 10%; hourly cost exceeds a configured threshold.

Actions

WARN, THROTTLE or BLOCK.

22. Database Design

Tables

users, applications, api_configurations, api_keys, usage_records, budgets, pricing_rules, anomalies, audit_logs.

Relationships

User → Applications → API Configurations and API Keys → Usage Records → Anomalies.

23. Database Indexes

Recommended

users.email; applications.owner_id; api_keys.key_hash; usage_records.application_id; usage_records.api_id; usage_records.created_at; composite application_id + created_at; anomalies.application_id + created_at.

Reason

These support authentication, dashboards, recent activity, usage analytics and anomaly queries.

24. Pagination

Rule

Never return thousands or millions of usage records in a single response.

Example

GET /usage?page=0&size=50.

Technology

Spring Data Pageable.

25. REST API Surface

Authentication

POST /api/v1/auth/register; POST /api/v1/auth/login; GET /api/v1/auth/me.

Applications

POST/GET/PUT/DELETE /api/v1/applications; GET /api/v1/applications/{id}.

APIs

POST/GET/PUT/DELETE /api/v1/apis; GET /api/v1/apis/{id}.

Keys

POST/GET /api/v1/api-keys; DELETE /api/v1/api-keys/{id}.

Usage

GET /api/v1/usage; GET /api/v1/usage/summary; GET /api/v1/usage/cost.

Anomalies

GET /api/v1/anomalies; POST /api/v1/anomalies/{id}/resolve.

Budgets

GET /api/v1/budgets; PUT /api/v1/budgets/{apid}.

Health

GET /actuator/health.

26. Standard API Responses

Success

```
{"success": true, "data": {}, "timestamp": "..."}

```

Error

```
{"success": false, "error": {"code": "RATE_LIMIT_EXCEEDED", "message": "Request limit exceeded"}, "timestamp": "..."}

```

27. Global Exception Handling

Technology

@RestControllerAdvice.

Handle

ResourceNotFoundException, validation failures, authentication/authorization errors, rate-limit errors, quota errors, budget errors, external API errors and timeouts.

Rule

Never expose Java stack traces to clients.

28. Java Package Architecture

Packages

auth, application, api, gateway, ratelimit, usage, cost, anomaly, cache, resilience, exception, config and common.

Principle

Separate controllers, services, repositories, entities, DTOs, policy components and infrastructure configuration.

29. Technology Stack

Backend

Java 21, Spring Boot, Spring Security, JWT, Spring Data JPA, Hibernate, PostgreSQL, Redis, Maven, Bucket4j, Resilience4j, OpenAPI/Swagger, JUnit 5, Mockito, Testcontainers, Docker, SLF4J/Logback and Spring Boot Actuator.

Frontend

React, TypeScript, Tailwind CSS and Recharts.

30. Frontend Screens

Screens

Login; Dashboard; API Management; API Details; Request Logs; Anomalies; Settings.

Dashboard metrics

Total requests, total cost, error rate, active APIs and active alerts.

Mobile

Responsive navigation, no horizontal overflow, readable tables, usable forms and clear success/error states.

31. Security Requirements

Secrets

Never commit JWT secrets, provider API keys, database passwords, Redis passwords or encryption keys.

Configuration

Use environment variables or a proper secret-management system.

Additional controls

Validate all input, enforce authorization, avoid sensitive logs and use least-privilege database credentials.

32. Threat Model

Brute-force traffic

Rate limiter.

Stolen API key

Revoke/block key and monitor abnormal usage.

Cost attack

Budget thresholds and automatic blocking.

Duplicate expensive operations

Idempotency where applicable.

Provider outage

Timeouts, bounded retries and circuit breaker.

33. Observability

Every request should answer

Who made it? Which application? Which API? When? What status? How long? How much did it cost? Was it cached? Was it rejected? Why was it rejected?

34. Testing Strategy

Unit tests

RateLimitService, CostCalculator, BudgetService, AnomalyDetector and QuotaService.

Integration tests

Spring Boot + PostgreSQL + Redis, preferably with Testcontainers.

API tests

401, 403, 404, 400, 429 and controlled 5xx behaviour.

Concurrency

Test simultaneous requests, especially around rate limits, quotas and budget exhaustion.

35. Prototype Demo Scenario

Scenario

Register a test API with a 100 requests/minute limit and a ■100 budget. Generate normal traffic, then simulate a runaway client.

Expected sequence

Traffic rises → anomaly detected → rate limiting activates → 429 responses appear → cost approaches threshold → warning at 90% → API blocks at 100% → dashboard reports protection activated.

Value

This demonstrates that API Sentinel is preventive control infrastructure, not merely an analytics dashboard.

36. Development Roadmap

Sprint 1

Java 21, Maven, Spring Boot, REST, PostgreSQL, JPA and basic project structure.

Sprint 2

Registration, login, JWT, Spring Security and roles.

Sprint 3

Application management, API configuration and API keys.

Sprint 4

Gateway proxy and external API forwarding.

Sprint 5

Rate limiting, quotas and spending caps.

Sprint 6

Timeouts, retries, circuit breaker and error handling.

Sprint 7

Redis caching and shared rate-limit state.

Sprint 8

Usage, cost, latency, errors and anomaly analytics.

Sprint 9

React dashboard, API management, logs, alerts and settings.

Sprint 10

Unit, integration, concurrency, security and failure testing.

37. Definition of Done

User workflow

Register → Login → Create application → Register API → Configure limits → Generate key → Send request → View usage → View cost → Trigger rate limit → Trigger budget protection → View anomaly → View logs.

Admin workflow

Disable API, revoke key, change limits, view anomalies and inspect system health.

38. Study Order

Java fundamentals

OOP, collections, exceptions, generics, streams, interfaces, records, enums, multithreading and concurrency.

Backend

HTTP, REST, JSON, status codes, DTOs, dependency injection and Spring Boot.

Database

SQL, joins, indexes, transactions, ACID, isolation levels, PostgreSQL and JPA/Hibernate.

Security

Authentication, authorization, JWT, password hashing, API keys, CORS, CSRF and secrets.

Distributed systems

Redis, caching, rate limiting, concurrency, idempotency, circuit breakers, retries and timeouts.

Production

Logging, monitoring, health checks, load testing, Docker and CI/CD.

39. Engineering Principle

Core principle

The project should demonstrate the complete request-control lifecycle: request → authentication → policy evaluation → rate limiting → quota → budget → cache → external service → resilience → usage tracking → cost calculation → observability.

Decision rule

Every feature should solve a real operational problem or strengthen the architecture. Avoid adding features merely to increase the technology count.

40. Reference Architecture

Layer	Responsibility
React Client	Dashboard, configuration, usage analytics, logs and alerts
Spring Boot API	REST endpoints, authentication, application management
Security	JWT authentication, API-key validation and authorization
Policy Engine	Rate limit, quota, budget and protection decisions
Gateway	Forward requests to configured external APIs
Redis	Caching and shared rate-limit state
PostgreSQL	Users, applications, configurations, keys, usage, budgets and anomalies
Resilience	Timeouts, retries and circuit breakers
Observability	Logs, metrics and health endpoints

Reference request flow

Client → Spring Boot Gateway → API Key Validation → Rate Limit → Quota → Budget → Redis Cache → External API → Timeout/Retry/Circuit Breaker → Usage Record → Cost Calculation → Response

41. Production Readiness Checklist

Area	Checklist
Frontend	No horizontal scroll; broken links fixed; mobile menu; favicon; page titles; meta descriptions; footer links; custom 404; dynamic cop
Backend	Rate limiting; API quotas; spending caps; global error handling; loading/error states; failed-request handling; timeout handling; ider
Security	Secrets outside Git; strong password hashing; JWT protection; API-key controls; authorization; safe logging; input validation; leas
Testing	Unit tests; integration tests; API tests; concurrency tests; failure tests; security tests; load tests.

Master specification: Treat this document as the source of truth for the prototype. Implement the MVP first, verify each feature with tests, and only then expand toward production-grade distributed infrastructure.